

The Ultimate Data Structures Reference Guide

Fenwick Trees, Segment Trees, Sparse Tables & More

Sahil Raj (After Dark)

Contents

1	Fenwick Tree (Binary Indexed Tree)	5
1.1	How It Works	5
1.2	Range Update, Point Query	6
1.3	Range Update, Range Query	6
1.4	2D Fenwick Tree	7
1.5	BIT for Order Statistics	7
1.6	Can BIT Do Min/Max? (Verdict + Workaround)	8
1.6.1	Exception: Prefix Max with Monotone Updates (Works!)	9
1.6.2	Why Decreasing Values Break BIT-Max	9
1.6.3	Verdict Table	10
2	Segment Tree	11
2.1	How It Works	11
2.2	Point Update, Range Query	11
2.2.1	Iterative Segment Tree (Faster, Cleaner)	11
2.2.2	Recursive Segment Tree (Extensible)	12
2.3	Supporting Different Operations	13
2.4	Custom Node: Multiple Values per Node	13
3	Lazy Propagation	15
3.1	The Problem: Range Updates are Slow	15
3.2	Range Add, Range Max	15
3.3	Range Assign, Range Min/Max	16
3.4	Combining Add and Assign Lazily	17
3.5	Generic Lazy Segment Tree Template	17
3.6	Lazy Propagation Cheat Sheet	19
4	Advanced Segment Trees	21
4.1	Persistent Segment Tree	21
4.1.1	K-th Smallest in Range [l, r] (Merge Sort Tree Alternative)	22
4.2	Merge Sort Tree	22
4.3	Segment Tree Beats (Ji Driver Segmentation)	23
4.4	Coordinate Compression + Segment Tree	24
4.5	Segment Tree Merge	25
4.5.1	Core Merge Algorithm	25
4.5.2	Application: k-th Smallest in Each Subtree	26
4.5.3	Application: Count Distinct Values per Subtree	26
4.5.4	Application: Merge with Frequency Histogram	27
5	Sparse Table & Sqrt Decomposition	29
5.1	Sparse Table	29
5.1.1	Sparse Table for GCD	30
5.2	Disjoint Sparse Table	30

5.3	Sqrt Decomposition	31
5.3.1	Point Update, Range Sum	31
5.3.2	Range Add, Point Query	31
5.4	Mo's Algorithm	32
5.4.1	Advanced Optimization: Hilbert Curve Sort	33
5.4.2	Advanced Variation: Mo's on Trees	33
5.4.3	Advanced Variation: Mo's with Rollback (Add-Only)	34
6	Applications & Patterns	37
6.1	Which Data Structure to Use?	37
6.2	Common Patterns	37
6.2.1	Offline with Coordinate Compression	37
6.2.2	Offline Segment Tree (Sweep Line)	38
6.2.3	Segment Tree + DP (Segment Tree DP)	38
6.3	Treap (Randomized BST)	38
6.4	Implicit Treap (Array with $O(\log n)$ Split/Merge)	40
6.5	Policy-Based Order Statistics Tree	40
6.6	Common Gotchas	41

Chapter 1

Fenwick Tree (Binary Indexed Tree)

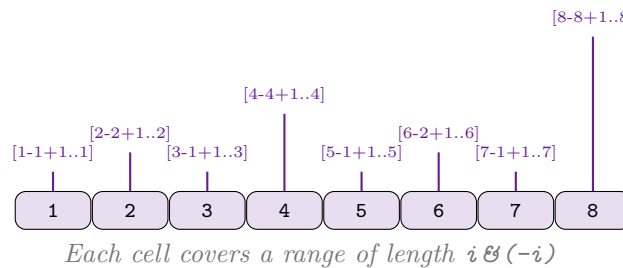
1.1 How It Works

How it Works

A **Fenwick Tree** (BIT) stores partial sums in a 1-indexed array `bit[]` where each index `i` is responsible for a range of length `i & (-i)` (the lowest set bit of `i`).

The lowbit trick: `i & (-i)` isolates the lowest set bit of `i`.

- `query(i)`: sum from 1 to `i`. Move *down* the tree: subtract lowbit repeatedly (`i -= i & -i`).
- `update(i, val)`: add `val` at position `i`. Move *up* the tree: add lowbit repeatedly (`i += i & -i`).



```
1 struct BIT {
2     int n;
3     vector<long long> bit;
4     BIT(int n) : n(n), bit(n + 1, 0) {}
5
6     // Add val to position i (1-indexed)
7     void update(int i, long long val) {
8         for (; i <= n; i += i & -i)
9             bit[i] += val;
10    }
11
12    // Sum of [1, i]
13    long long query(int i) {
14        long long s = 0;
15        for (; i > 0; i -= i & -i)
16            s += bit[i];
17        return s;
18    }
19
20    // Sum of [l, r]
21    long long query(int l, int r) { return query(r) - query(l - 1); }
22
23    // Build from array in O(n) instead of O(n log n)
24    void build(vector<long long>& a) {
25        for (int i = 1; i <= n; i++) {
26            bit[i] += a[i - 1];
```

```

27         int j = i + (i & -i);
28         if (j <= n) bit[j] += bit[i];
29     }
30 }
31 };

```

Complexity

Build: $\mathcal{O}(n)$. Point update: $\mathcal{O}(\log n)$. Prefix query: $\mathcal{O}(\log n)$. Space: $\mathcal{O}(n)$.

When to Use

Use BIT when: you need **point updates + prefix/range sum queries** and want a simpler, faster constant than a segment tree. BIT is $\sim 3\times$ faster in practice due to cache efficiency. **Limitation:** Only works for invertible operations (sum, XOR). Cannot do range-min/max queries directly.

1.2 Range Update, Point Query

How it Works

Use a **difference BIT**. To add `val` to range $[l, r]$: `update(l, val)` and `update(r+1, -val)`. Point query at `i` = prefix sum up to `i`.

```

1 struct BIT_RangeUpdate {
2     int n;
3     vector<long long> bit;
4     BIT_RangeUpdate(int n) : n(n), bit(n + 2, 0) {}
5
6     void rangeAdd(int l, int r, long long val) {
7         for (int i = l; i <= n; i += i & -i) bit[i] += val;
8         for (int i = r + 1; i <= n; i += i & -i) bit[i] -= val;
9     }
10
11     long long pointQuery(int i) { // value at position i
12         long long s = 0;
13         for (; i > 0; i -= i & -i) s += bit[i];
14         return s;
15     }
16 };

```

1.3 Range Update, Range Query

How it Works

Maintain **two BITs**: B_1 and B_2 . The prefix sum formula becomes:

$$\text{sum}(1, x) = B_1.\text{query}(x) \cdot x - B_2.\text{query}(x)$$

For range add $[l, r, +v]$: update B_1 with $+v$ at l and $-v$ at $r+1$; update B_2 with $+v(l-1)$ at l and $-v \cdot r$ at $r+1$.

```

1 struct BIT_RangeRangeSum {
2     int n;
3     vector<long long> b1, b2;
4     BIT_RangeRangeSum(int n) : n(n), b1(n+2,0), b2(n+2,0) {}
5
6     void add(vector<long long>& b, int i, long long v) {

```

```

7     for (; i <= n; i += i & -i) b[i] += v;
8 }
9 long long sum(vector<long long>& b, int i) {
10     long long s = 0;
11     for (; i > 0; i -= i & -i) s += b[i];
12     return s;
13 }
14
15 void rangeAdd(int l, int r, long long v) {
16     add(b1, l, v); add(b1, r+1, -v);
17     add(b2, l, v*(l-1)); add(b2, r+1, -v*r);
18 }
19
20 long long prefixSum(int x) { return sum(b1,x)*x - sum(b2,x); }
21 long long rangeSum(int l, int r) { return prefixSum(r) - prefixSum(l-1); }
22 };

```

1.4 2D Fenwick Tree

```

1 struct BIT2D {
2     int n, m;
3     vector<vector<long long>> bit;
4     BIT2D(int n, int m) : n(n), m(m), bit(n+1, vector<long long>(m+1, 0)) {}
5
6     void update(int x, int y, long long v) {
7         for (int i = x; i <= n; i += i & -i)
8             for (int j = y; j <= m; j += j & -j)
9                 bit[i][j] += v;
10    }
11
12    long long query(int x, int y) {
13        long long s = 0;
14        for (int i = x; i > 0; i -= i & -i)
15            for (int j = y; j > 0; j -= j & -j)
16                s += bit[i][j];
17        return s;
18    }
19
20    // Sum over rectangle [x1,y1] to [x2,y2] using inclusion-exclusion
21    long long query(int x1, int y1, int x2, int y2) {
22        return query(x2,y2) - query(x1-1,y2) - query(x2,y1-1) + query(x1-1,y1-1);
23    }
24 };

```

Complexity

2D BIT: $\mathcal{O}(\log n \cdot \log m)$ per update/query. Space: $\mathcal{O}(nm)$.

1.5 BIT for Order Statistics

How it Works

Coordinate-compress the values, then use a BIT over value space. $\text{query}(v)$ = count of elements $\leq v$. This gives $\mathcal{O}(\log n)$ rank and k -th element queries.

```

1 // Count inversions using BIT:  $\mathcal{O}(n \log n)$ 
2 long long countInversions(vector<int>& a) {
3     int n = a.size();
4     // Coordinate compress
5     vector<int> sorted = a; sort(sorted.begin(), sorted.end());

```

```

6   sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
7   auto compress = [&](int x) {
8       return lower_bound(sorted.begin(), sorted.end(), x) - sorted.begin() + 1;
9   };
10  BIT bit(sorted.size());
11  long long inv = 0;
12  for (int i = n - 1; i >= 0; i--) {
13      int v = compress(a[i]);
14      inv += bit.query(v - 1);    // how many elements to the right are smaller
15      bit.update(v, 1);
16  }
17  return inv;
18  }
19
20  // K-th smallest element using binary lifting on BIT
21  int kthSmallest(BIT& bit, int k) {
22      int pos = 0;
23      for (int pw = 1 << 20; pw; pw >>= 1)
24          if (pos + pw <= bit.n && bit.bit[pos + pw] < k) {
25              pos += pw;
26              k -= bit.bit[pos];
27          }
28      return pos + 1;
29  }

```

Key Insight / Optimization

Binary lifting on BIT: Walking down the BIT's binary structure to find the k -th element runs in $\mathcal{O}(\log n)$ without doing $\mathcal{O}(\log^2 n)$ binary search + prefix query. Standard trick in competitive programming.

Common Trap / Gotcha

BIT is 1-indexed. Passing $i=0$ to update/query silently does nothing (the loop condition $i > 0$ exits immediately). Always shift your array: `bit.update(i+1, ...)` when your data is 0-indexed.

1.6 Can BIT Do Min/Max? (Verdict + Workaround)

Common Trap / Gotcha

General range min/max with arbitrary point updates: NOT possible with a standard BIT.

The BIT works because its operation is an **invertible group**: to get the range sum $[l, r]$ you compute $\text{prefix}(r) - \text{prefix}(l - 1)$. Subtraction is the inverse of addition.

Min and Max have no inverse. Knowing $\text{min}(1..r)$ and $\text{min}(1..l-1)$ tells you nothing about $\text{min}(l..r)$ in general. You cannot “subtract” a prefix min to isolate a range min.

Conclusion: Use a **segment tree** for arbitrary-update range-min/max queries. A BIT simply cannot do it.

1.6.1 Exception: Prefix Max with Monotone Updates (Works!)

When to Use

BIT-max works ONLY when values only ever increase (you only call `update(i, v)` with $v \geq$ current value at i).

Under this constraint, `bit[i] = max` over some range and the query returns the correct prefix max.

Classic use: LIS in $\mathcal{O}(n \log n)$ — each coordinate is updated to the new LIS length, which only increases.

```

1 // BIT-Max: prefix maximum query,  $\mathcal{O}(\log n)$ 
2 // ONLY valid when values are non-decreasing at each position over time.
3 struct BITMax {
4     int n;
5     vector<int> bit;
6     BITMax(int n) : n(n), bit(n + 1, 0) {}
7
8     // Update: set position i to max(current, val) -- values only increase!
9     void update(int i, int val) {
10         for (; i <= n; i += i & -i)
11             bit[i] = max(bit[i], val); // max instead of +=
12     }
13
14     // Prefix max: max of [1, i]
15     int query(int i) {
16         int res = 0;
17         for (; i > 0; i -= i & -i)
18             res = max(res, bit[i]); // max instead of +=
19         return res;
20     }
21 };
22
23 // LIS length in  $\mathcal{O}(n \log n)$  using BIT-Max over coordinate space
24 int LIS(vector<int>& a) {
25     int n = a.size(), ans = 0;
26     // Coordinate compress
27     vector<int> sorted = a;
28     sort(sorted.begin(), sorted.end());
29     sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
30     int m = sorted.size();
31
32     BITMax bit(m);
33     for (int x : a) {
34         int v = lower_bound(sorted.begin(), sorted.end(), x) - sorted.begin() + 1;
35         int best = bit.query(v - 1) + 1; // longest IS ending with value < x
36         ans = max(ans, best);
37         bit.update(v, best); // update: value at position v is now 'best'
38     } // safe because best only increases for each v
39     return ans;
40 }

```

Complexity

BIT-Max: $\mathcal{O}(\log n)$ per update/query. LIS: $\mathcal{O}(n \log n)$ total.

1.6.2 Why Decreasing Values Break BIT-Max

```

1 // Counter-example: arbitrary updates break BIT-Max
2 // Array:   pos: 1  2  3  4
3 //           5  3  7  2    (initial)
4 // After bit.update(1, 5), bit.update(2, 3), bit.update(3, 7), bit.update(4, 2):
5 //   query(4) should return max(5,3,7,2) = 7 -- correct so far
6 // Now: bit.update(3, 1) -- decrease position 3 from 7 to 1
7 //   BIT-Max has no way to "undo" the 7 stored in parent nodes.

```

```

8 // query(4) still returns 7 even though position 3 is now 1.
9 // FIX: use a segment tree instead.

```

1.6.3 Verdict Table

Operation	BIT?	Condition	Alternative
Point update + prefix sum	✓Yes	Any updates	—
Point update + range sum	✓Yes	Any updates	—
Range update + range sum	✓Yes	Any updates	2-BIT trick
Point update + prefix max	✓Yes	Values only increase	BIT-Max above
Point update + prefix min	✓Yes	Values only decrease	BIT-Min (symmetric)
Point update + range max	×No	Any updates	Segment tree
Point update + range min	×No	Any updates	Segment tree
Range update + range max	×No	Any updates	Lazy seg tree

Chapter 2

Segment Tree

2.1 How It Works

How it Works

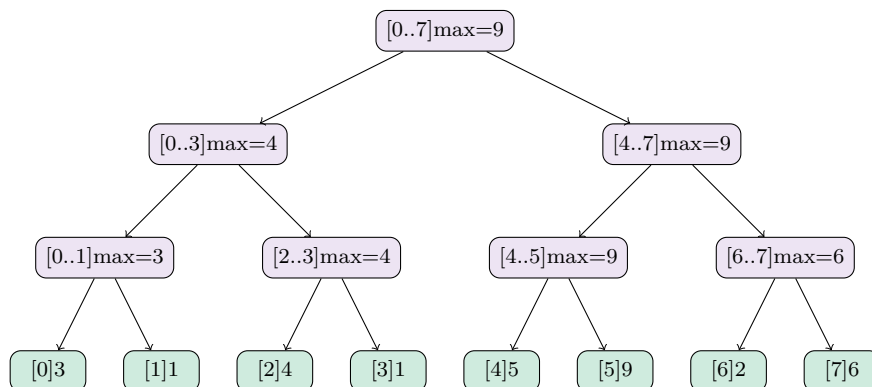
A **segment tree** is a complete binary tree over an array of size n .

Each node stores the answer for a contiguous range (segment) of the array.

Structure: Node 1 = root = range $[0, n - 1]$. Left child of node $i = 2i$. Right child = $2i + 1$.

Key idea: Any query range $[l, r]$ can be decomposed into $\mathcal{O}(\log n)$ non-overlapping segments stored in tree nodes.

Array size: Allocate $4n$ nodes to be safe (a complete binary tree over n leaves needs at most $4n$ nodes).



2.2 Point Update, Range Query

2.2.1 Iterative Segment Tree (Faster, Cleaner)

```
1 // Operation: XOR | identity = 0 | merge: a ^ b
2 struct SegTreeXOR {
3     int n;
4     vector<long long> tree;
5     SegTreeXOR(int n) : n(n), tree(2 * n, 0) {} // identity for XOR = 0
6
7     // Build from array: O(n)
8     void build(vector<int>& a) {
9         for (int i = 0; i < n; i++) tree[i + n] = a[i]; // leaves
10        for (int i = n - 1; i >= 1; i--) tree[i] = tree[2*i] ^ tree[2*i+1]; // XOR merge
11    }
12
13    // Point update: set position i to val, O(log n)
14    void update(int i, long long val) {
15        tree[i + n] = val;
16        for (i = (i + n) >> 1; i >= 1; i >>= 1)
```

```

17         tree[i] = tree[2*i] ^ tree[2*i+1]; // propagate XOR up
18     }
19
20     // Range XOR query [l, r) -- half-open interval, O(log n)
21     long long query(int l, int r) {
22         long long res = 0; // 0 is XOR identity
23         for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
24             if (l & 1) res ^= tree[l++]; // l is right child: XOR it in
25             if (r & 1) res ^= tree[--r]; // r is right child: XOR r-1 in
26         }
27         return res;
28     }
29 };
30 // To switch to MAX: replace '^' with 'max()', identity = LLONG_MIN
31 // To switch to MIN: replace '^' with 'min()', identity = LLONG_MAX
32 // To switch to GCD: replace '^' with '__gcd()', identity = 0

```

Complexity

Build: $\mathcal{O}(n)$. Update: $\mathcal{O}(\log n)$. Query: $\mathcal{O}(\log n)$. Space: $\mathcal{O}(n)$ (iterative uses exactly $2n$).

Key Insight / Optimization

Iterative vs recursive: The iterative version is $\sim 2\times$ faster due to no function-call overhead, better cache locality (bottom-up), and simpler code. Use it by default for **point update + range query**.

2.2.2 Recursive Segment Tree (Extensible)

The recursive version is easier to extend with lazy propagation or custom merge logic.

```

1 // Operation: MAX | identity = LLONG_MIN | merge: max(a, b)
2 // 0-based nodes: root = 0, left child = 2*nd+1, right child = 2*nd+2
3 struct SegTreeMax {
4     int n;
5     vector<long long> tree;
6     SegTreeMax(int n) : n(n), tree(4*n, LLONG_MIN) {} // identity for max = -inf
7
8     void build(vector<int>& a, int nd, int s, int e) {
9         if (s == e) { tree[nd] = a[s]; return; }
10        int mid = (s + e) / 2;
11        build(a, 2*nd+1, s, mid); // left child
12        build(a, 2*nd+2, mid+1, e); // right child
13        tree[nd] = max(tree[2*nd+1], tree[2*nd+2]); // MAX merge
14    }
15
16    void update(int nd, int s, int e, int i, long long val) {
17        if (s == e) { tree[nd] = val; return; }
18        int mid = (s + e) / 2;
19        if (i <= mid) update(2*nd+1, s, mid, i, val);
20        else update(2*nd+2, mid+1, e, i, val);
21        tree[nd] = max(tree[2*nd+1], tree[2*nd+2]); // pull up
22    }
23
24    long long query(int nd, int s, int e, int l, int r) {
25        if (r < s || e < l) return LLONG_MIN; // identity: max of empty range
26        if (l <= s && e <= r) return tree[nd];
27        int mid = (s + e) / 2;
28        return max(query(2*nd+1, s, mid, l, r), query(2*nd+2, mid+1, e, l, r));
29    }
30
31    void update(int i, long long val) { update(0, 0, n-1, i, val); } // root = 0
32    long long query(int l, int r) { return query(0, 0, n-1, l, r); }
33    // For MIN: swap max() for min(), identity = LLONG_MAX

```

34 };

2.3 Supporting Different Operations

The segment tree works for any **associative** merge operation. Just change the `merge` (the line that combines left and right child).

Operation	Merge (0-based: $L=2i+1$, $R=2i+2$)	Identity
Min	<code>tree[i] = min(tree[2i+1], tree[2i+2])</code>	$+\infty$
Max	<code>tree[i] = max(tree[2i+1], tree[2i+2])</code>	$-\infty$
XOR	<code>tree[i] = tree[2i+1] ^ tree[2i+2]</code>	0
GCD	<code>tree[i] = gcd(tree[2i+1], tree[2i+2])</code>	0
Product	<code>tree[i] = tree[2i+1] * tree[2i+2]</code>	1
AND / OR	<code>tree[i] = tree[2i+1] & tree[2i+2]</code>	all-1s / 0

Common Trap / Gotcha

Non-invertible operations (min, max, GCD): You cannot use a BIT for these — they require the segment tree. The BIT only supports operations where you can "undo" a prefix (i.e., subtraction for sum, XOR for XOR).

2.4 Custom Node: Multiple Values per Node

When you need to answer more complex queries (e.g., maximum subarray sum), store more info per node.

```

1 // Maximum subarray sum segment tree (Kadane's + D&C)
2 struct Node {
3     long long total;        // total sum of segment
4     long long maxPre;       // max prefix sum
5     long long maxSuf;       // max suffix sum
6     long long maxSub;       // max subarray sum (the answer)
7 };
8
9 Node combine(Node L, Node R) {
10     Node res;
11     res.total = L.total + R.total;
12     res.maxPre = max(L.maxPre, L.total + R.maxPre);
13     res.maxSuf = max(R.maxSuf, R.total + L.maxSuf);
14     res.maxSub = max({L.maxSub, R.maxSub, L.maxSuf + R.maxPre});
15     return res;
16 }
17
18 Node makeLeaf(long long v) { return {v, max(0LL, v), max(0LL, v), max(0LL, v)}; }
19
20 // 0-based: root=0, left=2nd+1, right=2nd+2
21 struct MaxSubSeg {
22     int n; vector<Node> tree;
23     MaxSubSeg(int n, vector<int>& a) : n(n), tree(4*n) { build(a,0,0,n-1); }
24     void build(vector<int>& a, int nd, int s, int e) {
25         if (s==e) { tree[nd]=makeLeaf(a[s]); return; }
26         int m=(s+e)/2;
27         build(a,2*nd+1,s,m); build(a,2*nd+2,m+1,e);
28         tree[nd]=combine(tree[2*nd+1],tree[2*nd+2]);
29     }
30     Node query(int nd,int s,int e,int l,int r) {
31         if (l<=s&&e<=r) return tree[nd];

```

```
32     int m=(s+e)/2;
33     if (r<=m) return query(2*nd+1,s,m,l,r);
34     if (l>m)  return query(2*nd+2,m+1,e,l,r);
35     return combine(query(2*nd+1,s,m,l,r), query(2*nd+2,m+1,e,l,r));
36 }
37 long long query(int l, int r) { return query(0,0,n-1,l,r).maxSub; } // root=0
38 };
```

Chapter 3

Lazy Propagation

3.1 The Problem: Range Updates are Slow

Without lazy propagation, a range update (e.g., add 5 to every element in $[l, r]$) requires updating every leaf in that range — $\mathcal{O}(n \log n)$ per update. That's too slow.

How it Works

Lazy propagation defers updates. When you need to update an entire segment $[s, e]$ that is *completely inside* the query range, instead of pushing the update all the way down to the leaves, **store the pending update in that node's lazy tag**.

Only **push the tag down** (*propagate*) when you actually need to visit the children (i.e., when the query range partially overlaps the node's range).

Result: Both range updates and range queries become $\mathcal{O}(\log n)$.

3.2 Range Add, Range Max

How it Works

Key difference from sum: When adding v to all elements in a range, the max of the range increases by v too. No need to multiply by segment length — just add v directly to the node's stored max. This makes the lazy tag simpler than for sum.

```
1 struct LazySegTree {
2     int n;
3     vector<long long> tree, lazy;
4     LazySegTree(int n) : n(n), tree(4*n, LLONG_MIN), lazy(4*n, 0) {}
5     // ~ identity for max
6
7     void build(vector<int>& a, int nd, int s, int e) {
8         if (s == e) { tree[nd] = a[s]; return; }
9         int m = (s + e) / 2;
10        build(a, 2*nd+1, s, m); build(a, 2*nd+2, m+1, e);
11        tree[nd] = max(tree[2*nd+1], tree[2*nd+2]); // MAX merge
12    }
13
14    // Push pending lazy add down to children
15    void pushDown(int nd) {
16        if (lazy[nd] == 0) return; // nothing pending
17        // For max: just add the lazy value -- NO length multiplication needed!
18        tree[2*nd+1] += lazy[nd]; lazy[2*nd+1] += lazy[nd];
19        tree[2*nd+2] += lazy[nd]; lazy[2*nd+2] += lazy[nd];
20        lazy[nd] = 0; // clear tag
21    }
```

```

22 // Range add: add val to every element in [l, r]
23 void update(int nd, int s, int e, int l, int r, long long val) {
24     if (r < s || e < l) return; // completely outside
25     if (l <= s && e <= r) { // completely inside: tag it
26         tree[nd] += val; // max of range shifts up by val
27         lazy[nd] += val; // defer to children
28         return;
29     }
30     pushDown(nd); // partial overlap: push first
31     int m = (s + e) / 2;
32     update(2*nd+1, s, m, l, r, val);
33     update(2*nd+2, m+1, e, l, r, val);
34     tree[nd] = max(tree[2*nd+1], tree[2*nd+2]); // pull up
35 }
36
37 // Range max query [l, r]
38 long long query(int nd, int s, int e, int l, int r) {
39     if (r < s || e < l) return LLONG_MIN; // identity for max
40     if (l <= s && e <= r) return tree[nd];
41     pushDown(nd); // push before visiting children
42     int m = (s + e) / 2;
43     return max(query(2*nd+1, s, m, l, r), query(2*nd+2, m+1, e, l, r));
44 }
45
46 void update(int l, int r, long long v) { update(0, 0, n-1, l, r, v); } // root=0
47 long long query(int l, int r) { return query(0, 0, n-1, l, r); }
48 // For MIN: swap max() for min(), init tree with LLONG_MAX, identity = LLONG_MAX
49 };
50

```

Complexity

Range update: $\mathcal{O}(\log n)$. Range query: $\mathcal{O}(\log n)$. Space: $\mathcal{O}(n)$.

Key Insight / Optimization

Range-add + range-max is simpler than range-add + range-sum.

Sum node: stores the *total* of all elements adding v to a segment of length L means the node's stored value increases by $v \times L$. You must multiply by length.

Max node: stores the *maximum* of all elements adding v shifts the maximum by exactly v , regardless of segment length. No multiplication needed.

This is why max/min lazy tags have no $(e-s+1)$ factor, making them shorter and less error-prone.

3.3 Range Assign, Range Min/Max

How it Works

When the update is **assignment** (set all elements in $[l, r]$ to v), the lazy tag semantics change slightly: a *new* assign overwrites any *old* assign. Use a sentinel (`LLONG_MIN`) to mean “no pending assign”.

```

1 const long long NONE = LLONG_MIN; // sentinel: no pending assignment
2
3 struct LazyAssignMin {
4     int n;
5     vector<long long> tree, lazy;
6     LazyAssignMin(int n) : n(n), tree(4*n, LLONG_MAX), lazy(4*n, NONE) {}
7
8     void pushDown(int nd) {
9         if (lazy[nd] == NONE) return;

```

```

10 // Assign lazy[nd] to both children (0-based: 2nd+1 and 2nd+2)
11 tree[2*nd+1] = lazy[nd]; lazy[2*nd+1] = lazy[nd];
12 tree[2*nd+2] = lazy[nd]; lazy[2*nd+2] = lazy[nd];
13 lazy[nd] = NONE;
14 }
15
16 void assign(int nd, int s, int e, int l, int r, long long val) {
17     if (r < s || e < l) return;
18     if (l <= s && e <= r) {
19         tree[nd] = val; lazy[nd] = val; return;
20     }
21     pushDown(nd);
22     int m = (s + e) / 2;
23     assign(2*nd+1, s, m, l, r, val); assign(2*nd+2, m+1, e, l, r, val);
24     tree[nd] = min(tree[2*nd+1], tree[2*nd+2]);
25 }
26
27 long long query(int nd, int s, int e, int l, int r) {
28     if (r < s || e < l) return LLONG_MAX;
29     if (l <= s && e <= r) return tree[nd];
30     pushDown(nd);
31     int m = (s + e) / 2;
32     return min(query(2*nd+1, s, m, l, r), query(2*nd+2, m+1, e, l, r));
33 }
34
35 void assign(int l, int r, long long v) { assign(0, 0, n-1, l, r, v); } // root=0
36 long long query(int l, int r) { return query(0, 0, n-1, l, r); }
37 };

```

3.4 Combining Add and Assign Lazily

When you need *both* range-assign and range-add, the lazy tag must compose correctly. Assign then add is fine: lazy = (assigned, added). But add then assign: the add is wiped out by the subsequent assign.

```

1 // Lazy tag: (assign_val, add_val). NONE = -1e18 means no assignment pending.
2 struct LazyTag {
3     long long asgn, add;
4     LazyTag() : asgn(NONE), add(0) {}
5 };
6
7 // Compose: apply tag 'b' on top of tag 'a' (a was already pending, then b comes)
8 LazyTag compose(LazyTag b, LazyTag a) {
9     if (b.asgn != NONE) return {b.asgn + b.add, 0}; // b's assign wipes a entirely
10    return {a.asgn == NONE ? NONE : a.asgn + b.add, a.add + b.add};
11 }
12
13 // Apply tag to a node covering 'len' elements
14 long long applyTag(LazyTag t, long long nodeVal, int len) {
15     if (t.asgn != NONE) return t.asgn * len + t.add * len;
16     return nodeVal + t.add * len;
17 }

```

3.5 Generic Lazy Segment Tree Template

A clean, reusable template where you define Node, Tag, merge, and apply:

```

1 template<typename Node, typename Tag>
2 struct GenericLazySeg {
3     int n;
4     vector<Node> tree;
5     vector<Tag> lazy;
6     function<Node(Node, Node)> merge; // how to combine two nodes
7     function<Node(Node, Tag, int)> apply; // apply tag to a node (len = segment length)

```

```

8      function<Tag(Tag, Tag)>                compose; // compose two tags: apply outer on inner
9      Node identity;      Tag tagId;          // identity values
10
11      GenericLazySeg(int n, Node id, Tag tid,
12          function<Node(Node,Node)> mg,
13          function<Node(Node,Tag,int)> ap,
14          function<Tag(Tag,Tag)> co)
15      : n(n), tree(4*n, id), lazy(4*n, tid),
16          merge(mg), apply(ap), compose(co), identity(id), tagId(tid) {}
17
18      void push(int nd, int s, int e) {
19          if (lazy[nd] == tagId) return;
20          int m = (s+e)/2;
21          tree[2*nd+1] = apply(tree[2*nd+1], lazy[nd], m-s+1);
22          tree[2*nd+2] = apply(tree[2*nd+2], lazy[nd], e-m);
23          lazy[2*nd+1] = compose(lazy[nd], lazy[2*nd+1]);
24          lazy[2*nd+2] = compose(lazy[nd], lazy[2*nd+2]);
25          lazy[nd] = tagId;
26      }
27
28      void update(int nd,int s,int e,int l,int r,Tag t){
29          if(r<s||e<l) return;
30          if(l<=s&&e<=r){tree[nd]=apply(tree[nd],t,e-s+1);lazy[nd]=compose(t,lazy[nd]);return
31      };}
32          push(nd,s,e); int m=(s+e)/2;
33          update(2*nd+1,s,m,l,r,t); update(2*nd+2,m+1,e,l,r,t);
34          tree[nd]=merge(tree[2*nd+1],tree[2*nd+2]);
35      }
36      Node query(int nd,int s,int e,int l,int r){
37          if(r<s||e<l) return identity;
38          if(l<=s&&e<=r) return tree[nd];
39          push(nd,s,e); int m=(s+e)/2;
40          return merge(query(2*nd+1,s,m,l,r),query(2*nd+2,m+1,e,l,r));
41      }
42      void update(int l,int r,Tag t){update(0,0,n-1,l,r,t);} // root=0
43      Node query(int l,int r){return query(0,0,n-1,l,r);}
44  };

```

Common Trap / Gotcha

The pushDown order matters. Always call *pushDown* *before* recursing into children, never after. If you forget and recurse first, the children may have stale data and your recomputed parent value will be wrong.

Common Trap / Gotcha

Lazy composition order. When composing tags: the *new* (incoming) tag is applied *on top of* the existing pending tag. Always think: “what happens if I first apply the old tag, then the new one?” and implement *compose* accordingly.

3.6 Lazy Propagation Cheat Sheet

Update	Query	Lazy tag	Apply to node
Range add v	Range max	add accumulates	<code>tree[nd] += v</code> (no length!)
Range add v	Range min	add accumulates	<code>tree[nd] += v</code> (no length!)
Range add v	Range sum	add accumulates	<code>tree[nd] += v*(e-s+1)</code>
Range assign v	Range max	asgn overwrites	<code>tree[nd] = v</code>
Range assign v	Range min	asgn overwrites	<code>tree[nd] = v</code>
Range assign v	Range sum	asgn overwrites	<code>tree[nd] = v*(e-s+1)</code>
Range add + assign	Any	Both fields, compose	Assign wins over old add
Range multiply v	Range max	mul accumulates	<code>tree[nd] *= v</code>

Chapter 4

Advanced Segment Trees

4.1 Persistent Segment Tree

When to Use

Why you need it:

You want to **query any historical version** of the segment tree (e.g., “what was the sum at time t ?”, “ k -th smallest in range $[l, r]$ ”).

Naive: Store a full copy of the tree at every version $\mathcal{O}(n)$ per version.

Persistent: Each update creates a new root and only modifies $\mathcal{O}(\log n)$ nodes (the path from root to the updated leaf). Old nodes are shared. $\mathcal{O}(\log n)$ per version.

Classic use: Offline k -th smallest in range $[l, r]$ using two versions of a sorted frequency tree.

```
1 struct PersistentSeg {
2     struct Node { int left, right; long long val; };
3     vector<Node> nodes;
4     vector<int> roots; // roots[i] = root of version i
5     int n;
6
7     PersistentSeg(int n) : n(n) { nodes.push_back({0, 0, 0}); roots.push_back(0); }
8
9     // Build initial version from array
10    int build(vector<int>& a, int s, int e) {
11        int idx = nodes.size(); nodes.push_back({0, 0, 0});
12        if (s == e) { nodes[idx].val = a[s]; return idx; }
13        int m = (s + e) / 2;
14        nodes[idx].left = build(a, s, m);
15        nodes[idx].right = build(a, m+1, e);
16        nodes[idx].val = nodes[nodes[idx].left].val + nodes[nodes[idx].right].val;
17        return idx;
18    }
19
20    // Create new version: copy path from prev root, update position i
21    int update(int prev, int s, int e, int i, long long val) {
22        int idx = nodes.size(); nodes.push_back(nodes[prev]); // copy node
23        if (s == e) { nodes[idx].val = val; return idx; }
24        int m = (s + e) / 2;
25        if (i <= m) nodes[idx].left = update(nodes[prev].left, s, m, i, val);
26        else nodes[idx].right = update(nodes[prev].right, m+1, e, i, val);
27        nodes[idx].val = nodes[nodes[idx].left].val + nodes[nodes[idx].right].val;
28        return idx;
29    }
30
31    long long query(int nd, int s, int e, int l, int r) {
32        if (r < s || e < l) return 0;
33        if (l <= s && e <= r) return nodes[nd].val;
34        int m = (s + e) / 2;
```

```

35     return query(nodes[nd].left, s, m, l, r) + query(nodes[nd].right, m+1, e, l, r);
36 }
37
38 // Add a new version by updating position i to val
39 void update(int i, long long val) {
40     roots.push_back(update(roots.back(), 0, n-1, i, val));
41 }
42 // Query range [l,r] in version v
43 long long query(int v, int l, int r) { return query(roots[v], 0, n-1, l, r); }
44 };

```

4.1.1 K-th Smallest in Range [l, r] (Merge Sort Tree Alternative)

```

1 // Build a persistent segment tree over VALUE space (frequency tree).
2 // Version i = after inserting the first i elements of the array.
3 // kth(l, r, k) = k-th smallest in a[l..r]
4 // Using roots[r] - roots[l-1] to isolate the [l,r] contribution.
5 int kthSmallest(PersistentSeg& ps, int leftRoot, int rightRoot, int s, int e, int k) {
6     if (s == e) return s; // s is the compressed value = answer
7     int m = (s + e) / 2;
8     long long leftCount = ps.nodes[ps.nodes[rightRoot].left].val
9                         - ps.nodes[ps.nodes[leftRoot].left].val;
10    if (k <= leftCount)
11        return kthSmallest(ps, ps.nodes[leftRoot].left, ps.nodes[rightRoot].left, s, m, k);
12    else
13        return kthSmallest(ps, ps.nodes[leftRoot].right, ps.nodes[rightRoot].right, m+1, e,
14                           k-leftCount);
15 }
16 // After coordinate-compressing values and building versions:
17 // kthSmallest(ps, roots[l-1], roots[r], 0, MAX_VAL-1, k)

```

Complexity

Space: $\mathcal{O}(n \log n)$ total nodes (each of n updates creates $\mathcal{O}(\log n)$ new nodes).

Query: $\mathcal{O}(\log n)$ per query. **Pre-allocate** the nodes vector to avoid reallocations.

Common Trap / Gotcha

Reallocation invalidates pointers/indices. Pre-allocate nodes with `nodes.reserve(n * 40)` before building. Each update creates at most $2 \log n$ nodes; $n = 10^5$ and $\log n \approx 17$ reserve $\sim 40n$ nodes.

4.2 Merge Sort Tree

When to Use

Why you need it:

You want to answer: “how many elements in $[l, r]$ are $\leq k$?” or “ k -th smallest in $[l, r]$?”

Each segment tree node stores a **sorted vector** of its range. Query = binary search at each of the $\mathcal{O}(\log n)$ nodes that cover $[l, r]$.

Offline alternative: Persistent segment tree is faster for k -th smallest but merge sort tree also supports “count $\leq k$ ” queries easily.

```

1 struct MergeSortTree {
2     int n;
3     vector<vector<int>> tree;
4     MergeSortTree(int n, vector<int>& a) : n(n), tree(4*n) { build(a, 1, 0, n-1); }
5
6     void build(vector<int>& a, int nd, int s, int e) {

```

```

7     if (s == e) { tree[nd] = {a[s]}; return; }
8     int m = (s+e)/2;
9     build(a, 2*nd, s, m); build(a, 2*nd+1, m+1, e);
10    merge(tree[2*nd].begin(), tree[2*nd].end(),
11          tree[2*nd+1].begin(), tree[2*nd+1].end(),
12          back_inserter(tree[nd]));          // merge sorted children
13 }
14
15 // Count elements <= k in [l, r]
16 int countLE(int nd, int s, int e, int l, int r, int k) {
17     if (r < s || e < l) return 0;
18     if (l <= s && e <= r)
19         return upper_bound(tree[nd].begin(), tree[nd].end(), k) - tree[nd].begin();
20     int m = (s+e)/2;
21     return countLE(2*nd, s, m, l, r, k) + countLE(2*nd+1, m+1, e, l, r, k);
22 }
23
24 int countLE(int l, int r, int k) { return countLE(1, 0, n-1, l, r, k); }
25
26 // K-th smallest in [l, r] via binary search over value space
27 int kthSmallest(int l, int r, int k) {
28     int lo = INT_MIN, hi = INT_MAX;
29     // Binary search on the sorted value space
30     // (better: use coordinate compression + binary lifting)
31     while (lo < hi) {
32         int mid = lo + (long long)(hi - lo) / 2;
33         if (countLE(l, r, mid) >= k) hi = mid;
34         else lo = mid + 1;
35     }
36     return lo;
37 }
38 };

```

Complexity

Build: $\mathcal{O}(n \log n)$ time and space (each element appears in $\mathcal{O}(\log n)$ nodes).

Query countLE: $\mathcal{O}(\log^2 n)$.

k -th smallest: $\mathcal{O}(\log^3 n)$ (outer binary search $\times$ inner query).

With coordinate compression k -th becomes $\mathcal{O}(\log^2 n)$.

4.3 Segment Tree Beats (Ji Driver Segmentation)

When to Use

Why you need it:

Standard lazy propagation can't handle: "set each $a_i = \min(a_i, v)$ for all $i \in [l, r]$ " (chmin update) with a range-max query efficiently.

Segment Tree Beats works in $\mathcal{O}((n + q) \log^2 n)$ amortized by tracking the *maximum* and *second maximum* at each node. The chmin update only propagates when $\text{val} > \text{second_max}$, so most nodes are skipped.

```

1 // 0-based: root=0, left=2nd+1, right=2nd+2
2 struct SegTreeBeats {
3     struct Node {
4         long long maxVal;    // maximum in segment
5         long long maxVal2;   // strict second maximum
6         long long sum;       // sum of all elements
7         int cntMax;          // count of elements equal to maxVal
8     };
9     int n; vector<Node> tree;
10 };

```

```

11 void pushUp(int nd) {
12     auto &L=tree[2*nd+1], &R=tree[2*nd+2], &P=tree[nd];
13     P.sum = L.sum + R.sum;
14     if (L.maxVal == R.maxVal) {
15         P.maxVal=L.maxVal; P.cntMax=L.cntMax+R.cntMax;
16         P.maxVal2=max(L.maxVal2, R.maxVal2);
17     } else if (L.maxVal > R.maxVal) {
18         P.maxVal=L.maxVal; P.cntMax=L.cntMax;
19         P.maxVal2=max(L.maxVal2, R.maxVal);
20     } else {
21         P.maxVal=R.maxVal; P.cntMax=R.cntMax;
22         P.maxVal2=max(L.maxVal, R.maxVal2);
23     }
24 }
25
26 // Apply chmin(val) to a node (only valid when val < current max)
27 void applyChmin(int nd, long long val) {
28     if (val >= tree[nd].maxVal) return;
29     tree[nd].sum -= (long long)(tree[nd].maxVal - val) * tree[nd].cntMax;
30     tree[nd].maxVal = val;
31 }
32
33 void pushDown(int nd) {
34     applyChmin(2*nd+1, tree[nd].maxVal);
35     applyChmin(2*nd+2, tree[nd].maxVal);
36 }
37
38 // Range chmin: for all i in [l,r], a[i] = min(a[i], val)
39 void updateChmin(int nd, int s, int e, int l, int r, long long val) {
40     if (r<s||e<l||val>=tree[nd].maxVal) return; // nothing to do
41     if (l<=s&&e<=r&&val>tree[nd].maxVal2) { // safe to apply directly
42         applyChmin(nd, val); return;
43     }
44     pushDown(nd); int m=(s+e)/2;
45     updateChmin(2*nd+1,s,m,l,r,val); updateChmin(2*nd+2,m+1,e,l,r,val);
46     pushUp(nd);
47 }
48
49 long long querySum(int nd, int s, int e, int l, int r) {
50     if (r<s||e<l) return 0;
51     if (l<=s&&e<=r) return tree[nd].sum;
52     pushDown(nd); int m=(s+e)/2;
53     return querySum(2*nd+1,s,m,l,r)+querySum(2*nd+2,m+1,e,l,r);
54 }
55 };

```

Complexity

$\mathcal{O}((n+q)\log^2 n)$ amortized. Supports: range chmin, range chmax, range add, range sum/max/min queries. One of the most powerful segment tree variants for interval scheduling and histogram problems.

4.4 Coordinate Compression + Segment Tree

How it Works

When values are large ($\leq 10^9$) but count is small ($n \leq 10^5$), compress them to $[0, n)$ before building the segment tree over value space.

```

1 % Compress values and build segment tree over value space
2 vector<int> compress(vector<int>& a) {
3     vector<int> sorted = a;
4     sort(sorted.begin(), sorted.end());

```

```

5 sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
6 for (int& x : a) x = lower_bound(sorted.begin(), sorted.end(), x) - sorted.begin();
7 return sorted; // sorted[compressed_value] = original value
8 }
9 // Now values in a[] are in [0, sorted.size()-1]. Build SegTree of that size.

```

4.5 Segment Tree Merge

When to Use

Why you need it:

In many tree DP problems, each node of the tree needs its own **frequency/value distribution** (a segment tree over value space). After computing children's distributions, you need to **merge** them into the parent.

Naive: Copy everything $\mathcal{O}(n)$ per merge $\mathcal{O}(n^2)$ total.

Segment Tree Merge: Recursively merge two sparse segment trees. Only recurse into a node if *both* children are non-null. Otherwise just link the existing node. Since each element contributes to $\mathcal{O}(\log n)$ nodes per merge, total work across all merges is $\mathcal{O}(n \log n)$ amortized.

Classic uses:

- **DSU on Tree:** Merge children's seg trees into parent, answer subtree queries.
- **k-th smallest in subtree:** Build a value-frequency seg tree per node, merge up.
- **Count inversions per subtree, frequency histograms, range union.**

4.5.1 Core Merge Algorithm

Each node of the *tree* (not the seg tree) owns a **sparse segment tree** over value space. Merging two sparse seg trees means: if both have a node at the same position, combine their values; if only one has a node, reuse it directly.

```

1 // Sparse (dynamic) segment tree node pool
2 struct Node {
3     int left, right; // children indices into pool (0 = null)
4     long long val; // e.g., count of elements in this range
5 };
6
7 const int MAXNODES = 6000000; // pre-allocate: n * log(MAXVAL) * 2
8 Node pool[MAXNODES];
9 int poolSize = 1; // 0 is the null node
10
11 int newNode() {
12     pool[poolSize] = {0, 0, 0};
13     return poolSize++;
14 }
15
16 // Merge two sparse seg trees rooted at u and v
17 // Destroys both u and v (u becomes the merged result).
18 // If you need both trees intact afterwards, clone nodes instead.
19 int merge(int u, int v) {
20     if (!u) return v; // u is null: result is v
21     if (!v) return u; // v is null: result is u
22     // Both non-null: create a combined node (reuse u)
23     pool[u].val += pool[v].val;
24     pool[u].left = merge(pool[u].left, pool[v].left);
25     pool[u].right = merge(pool[u].right, pool[v].right);
26     return u;
27 }
28
29 // Point insert: add 1 at position i in seg tree rooted at rt
30 int insert(int rt, int s, int e, int i) {
31     if (!rt) rt = newNode(); // create node on demand (sparse)

```

```

32 pool[rt].val++;
33 if (s == e) return rt;
34 int m = (s + e) / 2;
35 if (i <= m) pool[rt].left = insert(pool[rt].left, s, m, i);
36 else pool[rt].right = insert(pool[rt].right, m+1, e, i);
37 return rt;
38 }
39
40 // Query: k-th smallest element in seg tree rt over [s, e]
41 int kth(int rt, int s, int e, int k) {
42     if (s == e) return s; // reached a leaf: s is the answer (compressed value)
43     int m = (s + e) / 2;
44     long long leftCnt = pool[pool[rt].left].val;
45     if (k <= leftCnt) return kth(pool[rt].left, s, m, k);
46     else return kth(pool[rt].right, m+1, e, k - leftCnt);
47 }

```

4.5.2 Application: k-th Smallest in Each Subtree

```

1 // For each tree node u, build a seg tree over value space [0, MAXVAL).
2 // Then merge children's seg trees into parent bottom-up.
3 // After merge, kth(root[u], 0, MAXVAL-1, k) = k-th smallest value in subtree of u.
4
5 int root[100005]; // root[u] = root of seg tree for node u
6 int val[100005]; // value at each tree node (coordinate-compressed)
7 vector<int> tree[100005]; // adjacency list of the actual tree
8 int MAXVAL;
9
10 void dfs(int u, int par) {
11     root[u] = insert(0, 0, MAXVAL-1, val[u]); // build leaf seg tree for u itself
12     for (int v : tree[u]) {
13         if (v == par) continue;
14         dfs(v, u);
15         root[u] = merge(root[u], root[v]); // merge child into parent
16     }
17     // Now root[u] is the merged seg tree for the entire subtree of u
18     // Query: k-th smallest in subtree of u = kth(root[u], 0, MAXVAL-1, k)
19 }

```

Complexity

Total time: $\mathcal{O}(n \log \text{MAXVAL})$ for n inserts + $\mathcal{O}(n \log \text{MAXVAL})$ amortized for all merges.

Why merges are $\mathcal{O}(n \log \text{MAXVAL})$ total: Each element was inserted into exactly one leaf seg tree. Each merge step either links a node directly (free) or combines two non-null nodes (one gets destroyed). Each element can only be involved in $\mathcal{O}(\log \text{MAXVAL})$ combine steps before reaching the root.

Space: $\mathcal{O}(n \log \text{MAXVAL})$ nodes. For $n = 10^5$, $\text{MAXVAL} = 10^5$: $\approx 10^5 \times 17 \approx 1.7 \times 10^6$ nodes. Allocate $2\times$ to be safe.

4.5.3 Application: Count Distinct Values per Subtree

```

1 // After merging, pool[root[u]].val = count of distinct values in subtree of u
2 // (because insert increments val by 1 at each value node, merge sums them up)
3 // So if you want distinct count, track presence (insert 1, but cap at 1 per value).
4 // Alternatively: distinct = number of occupied leaves in the merged seg tree.
5
6 // Modified insert: only insert if value not already present (set semantics)
7 // This requires a separate "contains" check before inserting.
8 // Simpler: after merge, walk the tree and count non-zero leaves = distinct count.
9 // Or: use a different val semantics: pool[leaf].val = 1 if occupied, else 0.
10 // Then pool[root[u]].val after merge = number of distinct values in subtree of u.
11
12 // To count distinct values: just read pool[root[u]].val directly after merging

```

```
13 // (since each distinct value contributes exactly 1 to the root's val).
```

4.5.4 Application: Merge with Frequency Histogram

```
1 // Instead of counting distinct elements, track full frequency.
2 // Each merge sums frequencies. After merging subtree of u:
3 // Query prefix sum [0, k] in root[u] = count of elements <= k in subtree of u.
4
5 long long queryPrefix(int rt, int s, int e, int k) {
6     if (!rt || k < s) return 0;
7     if (e <= k) return pool[rt].val;
8     int m = (s + e) / 2;
9     return queryPrefix(pool[rt].left, s, m, k)
10        + queryPrefix(pool[rt].right, m+1, e, k);
11 }
12
13 // Range count in [lo, hi] for subtree of u:
14 // queryPrefix(root[u], 0, MAXVAL-1, hi) - queryPrefix(root[u], 0, MAXVAL-1, lo-1)
```

Key Insight / Optimization

Seg Tree Merge vs DSU on Tree (Sack):

- **DSU on Tree (Sack):** uses the heavy child's existing data structure and adds light children. Works with any container (map, multiset). $\mathcal{O}(n \log^2 n)$ with ordered sets.
- **Seg Tree Merge:** explicitly merges sparse seg trees. Better constant factor. Easier to query mid-merge. $\mathcal{O}(n \log n)$.
- **Use seg tree merge when:** you need complex range queries (k-th, count-LE, frequency sum) on merged subtrees *after* the merge is complete. Use Sack when the online queries during DFS are simpler.

Common Trap / Gotcha

Merge destroys both operands. The standard merge reuses one of the nodes and links into the other. After `root[u] = merge(root[u], root[v])`, `root[v]` is consumed and should not be used again. If you need both trees intact (e.g., for persistent queries), clone nodes during merge instead of reusing them.

Common Trap / Gotcha

Pool exhaustion. The node pool is statically sized. If you underestimate, you'll get a silent buffer overrun. Calculate: $(n_{\text{inserts}}) \times \lceil \log_2 \text{MAXVAL} \rceil \times 2$ (factor 2 for safety). Pre-check `poolSize < MAXNODES` during insert/merge for debugging.

Chapter 5

Sparse Table & Sqrt Decomposition

5.1 Sparse Table

When to Use

Why you need it:

You have a **static array** (no updates) and need to answer **range min/max** (or any idempotent operation) queries in $\mathcal{O}(1)$. A segment tree does $\mathcal{O}(\log n)$ per query; a sparse table does $\mathcal{O}(1)$.

Idempotent means: $f(x, x) = x$. Works for: min, max, GCD, bitwise AND/OR.

Does NOT work for sum (not idempotent, since $x + x \neq x$). Use prefix sums or BIT for that.

How it Works

How it works:

Precompute $\text{sp}[k][i] = \text{answer for the range } [i, i + 2^k - 1]$.

For RMQ query $[l, r]$: find $k = \lfloor \log_2(r - l + 1) \rfloor$, then:

$$\text{ans} = \min(\text{sp}[k][l], \text{sp}[k][r - 2^k + 1])$$

The two overlapping ranges of length 2^k cover $[l, r]$ completely. Since min is idempotent, overlapping is fine.

```
1 struct SparseTable {
2     int n, LOG;
3     vector<vector<int>> sp;
4     vector<int> lg; // lg[i] = floor(log2(i))
5
6     SparseTable(vector<int>& a) {
7         n = a.size();
8         LOG = __lg(n) + 1; // number of levels needed
9         sp.assign(LOG, vector<int>(n));
10        lg.resize(n + 1);
11        lg[1] = 0;
12        for (int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
13
14        sp[0] = a; // level 0: ranges of length 1
15        for (int k = 1; k < LOG; k++)
16            for (int i = 0; i + (1<<k) <= n; i++)
17                sp[k][i] = min(sp[k-1][i], sp[k-1][i + (1<<(k-1))]);
18    }
19
20    // O(1) range minimum query [l, r]
21    int query(int l, int r) {
22        int k = lg[r - l + 1];
23        return min(sp[k][l], sp[k][r - (1<<k) + 1]);
24    }
```

```
25 };
```

Complexity

Build: $\mathcal{O}(n \log n)$ time and space. Query: $\mathcal{O}(1)$.

Use **when**: static array, many range min/max queries, speed critical.

5.1.1 Sparse Table for GCD

```
1 // Identical structure, just change min to gcd
2 // GCD(x, x) = x, so it's idempotent
3 sp[k][i] = __gcd(sp[k-1][i], sp[k-1][i + (1<<(k-1))]);
4 int queryGCD(int l, int r) {
5     int k = lg[r - l + 1];
6     return __gcd(sp[k][l], sp[k][r - (1<<k) + 1]);
7 }
```

5.2 Disjoint Sparse Table

How it Works

Extends sparse table to **non-idempotent** operations (like sum, product, XOR) while maintaining $\mathcal{O}(1)$ queries. Stores prefix/suffix answers at each level without overlapping ranges.

Build: $\mathcal{O}(n \log n)$. Query: $\mathcal{O}(1)$.

```
1 struct DisjointSparseTable {
2     int n, LOG;
3     vector<vector<long long>> table;
4
5     DisjointSparseTable(vector<int>& a) {
6         n = a.size(); LOG = 1;
7         while ((1 << LOG) < n) LOG++;
8         table.assign(LOG + 1, vector<long long>(n, 0));
9         table[0] = vector<long long>(a.begin(), a.end());
10
11         for (int k = 1; k <= LOG; k++) {
12             int len = 1 << k;
13             for (int i = 0; i < n; i += len) {
14                 int mid = i + len / 2;
15                 // Build suffix sums to the left of mid
16                 table[k][mid - 1] = table[0][mid - 1];
17                 for (int j = mid - 2; j >= i; j--)
18                     table[k][j] = table[k][j+1] + table[0][j]; // or any operation
19                 // Build prefix sums to the right of mid
20                 table[k][mid] = table[0][min(mid, n-1)];
21                 for (int j = mid + 1; j < min(i + len, n); j++)
22                     table[k][j] = table[k][j-1] + table[0][j];
23             }
24         }
25     }
26
27     long long query(int l, int r) {
28         if (l == r) return table[0][l];
29         int k = __lg(1 ^ r); // highest differing bit
30         return table[k][l] + table[k][r]; // suffix from l + prefix to r
31     }
32 };
```

5.3 Sqrt Decomposition

When to Use

Why you need it:

A powerful, flexible technique that gives $\mathcal{O}(\sqrt{n})$ per query/update. While slower than a segment tree's $\mathcal{O}(\log n)$, it is incredibly versatile. You can store arbitrary data structures inside each block (e.g., sorting the block, or keeping a 'std::set'), allowing operations that segment trees simply cannot do.

Idea: Split the array into blocks of size $B \approx \sqrt{n}$. For a range query: handle partial left/right blocks element-by-element (at most $2B$ elements), and full middle blocks in $\mathcal{O}(1)$ each (at most n/B blocks).

5.3.1 Point Update, Range Sum

```

1 struct SqrtDecomp {
2     int n, B;
3     vector<long long> a, block;
4
5     SqrtDecomp(vector<int>& arr) {
6         n = arr.size(); B = max(1, (int)sqrt(n));
7         a.assign(arr.begin(), arr.end());
8         block.assign((n + B - 1) / B, 0);
9         for (int i = 0; i < n; i++) block[i / B] += a[i];
10    }
11
12    void update(int i, long long val) {
13        block[i / B] += val - a[i];           // update block aggregate
14        a[i] = val;                           // update actual element
15    }
16
17    long long query(int l, int r) {
18        long long res = 0;
19        int lb = l / B, rb = r / B;
20        if (lb == rb) {                        // same block: brute force
21            for (int i = l; i <= r; i++) res += a[i];
22            return res;
23        }
24        for (int i = l; i < (lb+1)*B; i++) res += a[i]; // partial left block
25        for (int b = lb+1; b < rb; b++) res += block[b]; // full middle blocks
26        for (int i = rb*B; i <= r; i++) res += a[i];   // partial right block
27        return res;
28    }
29 };

```

5.3.2 Range Add, Point Query

Instead of aggregating, the block array acts as a **lazy tag**.

```

1 // For range add [l,r,v]:
2 // Partial blocks: a[i] += v directly.
3 // Full middle blocks: block_lazy[b] += v.
4 // For point query(i): return a[i] + block_lazy[i / B].

```

Complexity

$\mathcal{O}(\sqrt{n})$ per update and query. $\mathcal{O}(n)$ build.

Why $B = \sqrt{n}$? A query takes $\mathcal{O}(B)$ for partial blocks and $\mathcal{O}(n/B)$ for full blocks. Minimizing $B + n/B$ gives $B \approx \sqrt{n}$.

Feasibility: For $n = 10^5$, $\sqrt{n} \approx 316$. With $q = 10^5$ queries, 3×10^7 operations easily pass in 1 second.

5.4 Mo's Algorithm

When to Use

Why you need it:

Offline range queries where you can add/remove elements from the current window in $\mathcal{O}(1)$ each. Mo's processes queries in a smart order so the total number of add/remove operations is drastically minimized.

Classic uses: Count distinct elements in range. Mode/frequency queries. Count pairs summing to K . Problems where segment tree merge is too slow.

Constraint: All queries must be known upfront (offline).

How it Works

The Magic Sorting Order:

Sort queries into **blocks** based on their left endpoint L/B . Within the same block, sort by their right endpoint R .

The Zigzag trick: Alternate the R sort direction (ascending in even blocks, descending in odd blocks) so the R pointer sweeps left instead of resetting all the way back to 0.

```

1 struct Mo {
2     int n, B;
3     struct Query { int l, r, idx; };
4
5     void solve(int n, int q, vector<Query>& queries, vector<int>& a, vector<long long>& ans)
6     {
7         // Optimal block size when Q != N is N / sqrt(Q)
8         B = max(1, (int)(n / sqrt(max(1, q))));
9
10        sort(queries.begin(), queries.end(), [&](const Query& x, const Query& y) {
11            int bx = x.l / B, by = y.l / B;
12            if (bx != by) return bx < by;
13            return (bx & 1) ? (x.r > y.r) : (x.r < y.r); // zigzag for speed
14        });
15
16        // Maintain current window [curL, curR]
17        int curL = 0, curR = -1;
18        long long curAns = 0;
19        vector<int> freq(200005, 0); // State array
20        int distinct = 0;
21
22        // O(1) transitions
23        auto add = [&](int i) {
24            if (freq[a[i]]++ == 0) distinct++;
25        };
26        auto remove = [&](int i) {
27            if (--freq[a[i]] == 0) distinct--;
28        };
29
30        for (auto& qry : queries) {
31            // ALWAYS expand before shrinking to avoid negative window sizes
32            while (curL > qry.l) add(--curL);
33            while (curR < qry.r) add(++curR);
34            while (curL < qry.l) remove(curL++);
35            while (curR > qry.r) remove(curR--);
36            ans[qry.idx] = distinct;
37        }
38    };

```

Complexity

Complexity Proof: why is it $\mathcal{O}(N\sqrt{Q})$?

- **Right pointer:** Within a block, R sweeps from 1 to N . There are N/B blocks, so R moves $\mathcal{O}(N \times \frac{N}{B}) = \mathcal{O}(N^2/B)$ times total.
- **Left pointer:** Between queries in the same block, L moves at most $\mathcal{O}(B)$ times. Over Q queries, L moves $\mathcal{O}(Q \times B)$ times total.
- **Total movement:** $\mathcal{O}(\frac{N^2}{B} + Q \times B)$. Differentiating with respect to B gives the optimal block size $B = \frac{N}{\sqrt{Q}}$, resulting in exactly $\mathcal{O}(N\sqrt{Q})$ operations.

5.4.1 Advanced Optimization: Hilbert Curve Sort

Key Insight / Optimization

For very tight time limits, sorting queries by their 2D position (L, R) along a **Hilbert Curve** provides roughly a **2x speedup** over standard Mo's blocks. It maximizes spatial locality for the pointers.

```

1 inline int64_t hilbertOrder(int x, int y, int pow, int rotate) {
2     if (pow == 0) return 0;
3     int hpow = 1 << (pow - 1);
4     int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
5     seg = (seg + rotate) & 3;
6     const int rotateDelta[4] = {3, 0, 0, 1};
7     int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
8     int nrot = (rotate + rotateDelta[seg]) & 3;
9     int64_t subSquareSize = int64_t(1) << (2 * pow - 2);
10    int64_t ans = seg * subSquareSize;
11    int64_t add = hilbertOrder(nx, ny, pow - 1, nrot);
12    ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
13    return ans;
14 }
15 // Usage: Precompute gry.ord = hilbertOrder(gry.l, gry.r, 21, 0)
16 // Sort queries simply by 'gry.ord < other.ord'.

```

Common Trap / Gotcha

Mo's does NOT support updates. For updates, use **Mo's with modifications** (3D sorting: block of l , block of time, then r) in $\mathcal{O}(n^{5/3})$ per query. Also cannot be used for problems where add and remove are hard to implement (e.g., gcd of a set does not support remove in $\mathcal{O}(1)$).

5.4.2 Advanced Variation: Mo's on Trees

When to Use

Why you need it:

You have offline queries asking for properties of the **simple path between node U and node V** on a tree.

Idea: Flatten the tree into an array using an **Euler Tour**. Then, path queries become standard subarray queries, allowing you to run standard Mo's algorithm!

How it Works

Tree Flattening Rules:

1. Run a DFS. Add node u to the array when you *enter* it ($in[u]$), and add it again when you *leave* it ($out[u]$). The array size will be $2N$.
2. To query the path between U and V (assume $in[U] < in[V]$):
 - **Case 1 (U is ancestor of V):** The path is the contiguous range $[in[U], in[V]]$.
 - **Case 2 (U is not ancestor):** The path is the contiguous range $[out[U], in[V]]$, **PLUS** their LCA.
3. **The toggle rule:** If a node appears exactly **once** in the queried range, it is **ON** the path. If it appears **twice** (both entered and left), it is **OFF** the path.

```

1 // The "add" and "remove" functions become a single "toggle" function
2 vector<bool> in_window(N + 1, false);
3
4 auto toggle = [&](int node) {
5     if (in_window[node]) {
6         // Node is currently in window, so we are removing it
7         remove_element(val[node]);
8     } else {
9         // Node is not in window, so we are adding it
10        add_element(val[node]);
11    }
12    in_window[node] = !in_window[node];
13 };
14
15 // Inside Mo's loop (Case 2 example):
16 while (curL > q.l) toggle(euler_array[--curL]);
17 // ... standard 4 while loops ...
18
19 // If Case 2, manually toggle the LCA to include it in the answer
20 if (q.lca != 0) toggle(q.lca);
21 ans[q.idx] = current_answer;
22 if (q.lca != 0) toggle(q.lca); // toggle back to remove it

```

5.4.3 Advanced Variation: Mo's with Rollback (Add-Only)

When to Use

Why you need it:

Standard Mo's requires both an $\mathcal{O}(1)$ 'add' and an $\mathcal{O}(1)$ 'remove'. But what if you are maintaining a **DSU** or tracking the **Maximum**? Adding to a DSU is easy; removing from a DSU is extremely hard.

Idea: We can modify Mo's so that we **never call remove**. We only call 'add', and a special 'rollback' function that undoes the last X additions using a history stack.

How it Works

How Rollback Mo's Works:

1. Sort queries standardly by block of L , then ascending by R . (No zigzag! R must strictly increase within a block).
2. For a fixed block B , process all queries whose L falls inside B .
3. **Small queries** (both L and R in the same block) are brute-forced in $\mathcal{O}(\text{block_size})$ time.
4. **Large queries:** Set R_{ptr} to the right edge of block B . It only moves right, so we only **add** elements.
5. Set L_{ptr} to the right edge of block B . Move it left to the query's L , **adding** elements along the way.
6. Record the answer.
7. **Rollback** the L_{ptr} back to the right edge of block B by popping history off a stack. (This is an undo, not a remove!).

Complexity

Because R_{ptr} only moves right within a block, and L_{ptr} only moves within a single block before rolling back, the time complexity remains identical to standard Mo's: $\mathcal{O}(N\sqrt{Q})$. It perfectly handles offline queries for non-invertible data structures like DSU!

Chapter 6

Applications & Patterns

6.1 Which Data Structure to Use?

Operation	Updates?	Best DS	Time/query	Note
Range sum	Point update	BIT	$\mathcal{O}(\log n)$	Simplest + fastest
Range sum	Range update	Lazy seg tree	$\mathcal{O}(\log n)$	Or 2-BIT trick
Range min/max	None	Sparse table	$\mathcal{O}(1)$	Static only
Range min/max	Point update	Seg tree	$\mathcal{O}(\log n)$	Can't use BIT
Range min/max	Range update	Lazy seg tree	$\mathcal{O}(\log n)$	
Range GCD	None	Sparse table	$\mathcal{O}(1)$	Idempotent
Range GCD	Point update	Seg tree	$\mathcal{O}(\log n)$	
k -th smallest	Online	Persist. seg	$\mathcal{O}(\log n)$	After coord compress
k -th smallest	Offline	Merge sort tree	$\mathcal{O}(\log^2 n)$	Simpler
Count distinct	Offline	Mo's algo	$\mathcal{O}(\sqrt{n})$	Per query
Chmin/chmax	Range	Seg tree beats	$\mathcal{O}(\log^2 n)$	Amortized
Historical max	Range	Persist. segtree	$\mathcal{O}(\log n)$	
2D point update	Point	2D BIT	$\mathcal{O}(\log^2 n)$	

6.2 Common Patterns

6.2.1 Offline with Coordinate Compression

A huge fraction of hard DS problems use this pattern:

1. Read all values. Collect and sort all distinct values.
2. Map each value to its rank (0-indexed position in sorted list).
3. Build your DS over ranks, not original values.

```
1 // Template for coordinate compression + BIT for count queries
2 vector<int> vals; // collect all values first
3 sort(vals.begin(), vals.end());
4 vals.erase(unique(vals.begin(), vals.end()), vals.end());
5 auto rank_ = [&](int x) {
6     return (int)(lower_bound(vals.begin(), vals.end(), x) - vals.begin());
7 };
```

```

8 BIT bit(vals.size());
9 // Now: bit.update(rank_(x), 1) = insert x; bit.query(rank_(y)) = count <= y

```

6.2.2 Offline Segment Tree (Sweep Line)

Process events in sorted order, maintaining a segment tree over one dimension.

```

1 // Count rectangles containing each point: offline sweep
2 // Events sorted by x-coordinate:
3 // - at x=x1: add y-range [y1,y2] with weight +1
4 // - at x=x2+1: add y-range [y1,y2] with weight -1
5 // Query: point (px, py) query at x=px, position y=py in the seg tree
6 struct Event { int x, y1, y2, val; };
7 // Sort events + queries by x, process with lazy seg tree

```

6.2.3 Segment Tree + DP (Segment Tree DP)

Use a segment tree to accelerate DP transitions where the recurrence is a range query.

```

1 // Example: dp[i] = max(dp[j] + cost(j,i)) for j < i where cost is a simple function
2 // Maintain a segment tree where node j stores dp[j] + some prefix function of j.
3 // dp[i] = max over a range in the seg tree.
4
5 // LIS in O(n log n) using BIT/seg tree:
6 // dp[i] = length of longest increasing subsequence ending at a[i]
7 // = 1 + max(dp[j]) for all j < i where a[j] < a[i]
8 // After coordinate compressing values, query max in [0, a[i]-1], update at a[i]
9 vector<int> lis_bit; // BIT storing max LIS length at each value
10 int lisLength(vector<int>& a) {
11     int n = a.size(), ans = 0;
12     // Coordinate compress a
13     vector<int> sorted = a; sort(sorted.begin(), sorted.end());
14     sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
15     int m = sorted.size();
16     lis_bit.assign(m + 1, 0);
17     // BIT max query/update
18     auto query = [&](int i) { int s=0; for(;;i>0;i-=i&-1) s=max(s,lis_bit[i]); return s; };
19     auto update = [&](int i,int v){ for(;;i<=m;i+=i&-1) lis_bit[i]=max(lis_bit[i],v); };
20     for (int x : a) {
21         int v = lower_bound(sorted.begin(),sorted.end(),x)-sorted.begin()+1;
22         int best = query(v - 1) + 1;
23         update(v, best);
24         ans = max(ans, best);
25     }
26     return ans;
27 }

```

6.3 Treap (Randomized BST)

When to Use

Why you need it:

A **treap** is a BST + heap simultaneously. Each node has a *key* (BST property) and a random *priority* (heap property). This keeps the tree balanced with high probability.

Use over segment tree when: You need **split** and **merge** by key in $\mathcal{O}(\log n)$, which lets you implement: order statistics, range reversal (implicit treap), insertion/deletion at arbitrary positions.

```

1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 struct Treap {
4     struct Node {

```

```

5     int key, pri, sz;
6     long long sum;                // any aggregate
7     Node *l, *r;
8     Node(int k) : key(k), pri(rng()), sz(1), sum(k), l(nullptr), r(nullptr) {}
9 };
10
11 int sz(Node* t) { return t ? t->sz : 0; }
12 long long sum(Node* t) { return t ? t->sum : 0; }
13
14 void pull(Node* t) {
15     if (!t) return;
16     t->sz = 1 + sz(t->l) + sz(t->r);
17     t->sum = t->key + sum(t->l) + sum(t->r);
18 }
19
20 // Split by key: l gets keys <= key, r gets keys > key
21 pair<Node*, Node*> split(Node* t, int key) {
22     if (!t) return {nullptr, nullptr};
23     if (t->key <= key) {
24         auto [l, r] = split(t->r, key);
25         t->r = l; pull(t); return {t, r};
26     } else {
27         auto [l, r] = split(t->l, key);
28         t->l = r; pull(t); return {l, t};
29     }
30 }
31
32 // Merge two treaps (all keys in l <= all keys in r)
33 Node* merge(Node* l, Node* r) {
34     if (!l) return r; if (!r) return l;
35     if (l->pri > r->pri) { l->r = merge(l->r, r); pull(l); return l; }
36     else { r->l = merge(l, r->l); pull(r); return r; }
37 }
38
39 Node* root = nullptr;
40
41 void insert(int key) {
42     auto [l, r] = split(root, key - 1);
43     root = merge(merge(l, new Node(key)), r);
44 }
45
46 void erase(int key) {
47     auto [l, mid] = split(root, key - 1);
48     auto [m, r] = split(mid, key);
49     // m has exactly one node with key 'key' (if it existed)
50     root = merge(l, r);
51 }
52
53 long long querySum(int lo, int hi) {
54     auto [l, mid] = split(root, lo - 1);
55     auto [m, r] = split(mid, hi);
56     long long res = sum(m);
57     root = merge(l, merge(m, r));
58     return res;
59 }

```

Complexity

All operations: $\mathcal{O}(\log n)$ expected. No worst-case guarantee (randomized), but failure probability is negligible.

6.4 Implicit Treap (Array with $\mathcal{O}(\log n)$ Split/Merge)

How it Works

Replace the BST key with **implicit key = subtree size**. This lets you treat the treap as a dynamic array supporting $\mathcal{O}(\log n)$ split/merge at any position, and lazy range operations (including **range reversal**).

```

1 struct ImplicitTreap {
2     struct Node {
3         int pri, sz; long long val, sum;
4         bool rev; // lazy reversal flag
5         Node *l, *r;
6         Node(long long v) : pri(rng()), sz(1), val(v), sum(v), rev(false), l(nullptr), r(
7             nullptr) {}
8     };
9     int sz(Node* t) { return t ? t->sz : 0; }
10    void pull(Node* t) { if(t){ t->sz=1+sz(t->l)+sz(t->r); t->sum=t->val+(t->l?t->l->sum:0)
11        +(t->r?t->r->sum:0); } }
12    void applyRev(Node* t) { if(t){ swap(t->l,t->r); t->rev^=1; } }
13    void push(Node* t) { if(t&& t->rev){ applyRev(t->l); applyRev(t->r); t->rev=false; } }
14
15    // Split first k elements into l, rest into r
16    pair<Node*,Node*> split(Node* t, int k) {
17        if (!t) return {nullptr, nullptr};
18        push(t);
19        if (sz(t->l) >= k) { auto[l,r]=split(t->l,k); t->l=r; pull(t); return{l,t}; }
20        else { auto[l,r]=split(t->r,k-sz(t->l)-1); t->r=l; pull(t); return{t,r}; }
21    }
22    Node* merge(Node* l, Node* r) {
23        push(l); push(r);
24        if (!l) return r; if (!r) return l;
25        if (l->pri > r->pri) { l->r=merge(l->r,r); pull(l); return l; }
26        else { r->l=merge(l,r->l); pull(r); return r; }
27    }
28
29    Node* root = nullptr;
30    void insert(int pos, long long val) { // insert at position pos (0-indexed)
31        auto[l,r]=split(root,pos); root=merge(merge(l,new Node(val)),r);
32    }
33    void reverse(int l, int r) { // reverse subarray [l,r]
34        auto[L,mid]=split(root,l); auto[M,R]=split(mid,r-l+1);
35        applyRev(M); root=merge(L,merge(M,R));
36    }
37    long long querySum(int l, int r) {
38        auto[L,mid]=split(root,l); auto[M,R]=split(mid,r-l+1);
39        long long res=M?M->sum:0; root=merge(L,merge(M,R)); return res;
40    }
41 };

```

6.5 Policy-Based Order Statistics Tree

```

1 // GNU PBDS: O(log n) order statistics
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4 using namespace __gnu_pbds;
5
6 // Ordered set: supports order_of_key and find_by_order in O(log n)
7 using ordered_set = tree<int, null_type, less<int>,
8     rb_tree_tag, tree_order_statistics_node_update>;
9
10 ordered_set s;
11 s.insert(5); s.insert(2); s.insert(8);
12 s.order_of_key(5); // 1 -- number of elements < 5
13 *s.find_by_order(1); // 5 -- 0-indexed 1st smallest element

```

```

14
15 // For duplicate keys, use pair<int,int> with a unique counter:
16 ordered_set ms;
17 int cnt = 0;
18 ms.insert({5, cnt++}); ms.insert({5, cnt++}); // two 5s

```

Complexity

All operations: $\mathcal{O}(\log n)$. Use instead of a segment tree for dynamic order statistics when elements are inserted/deleted and you need k -th element or rank.

6.6 Common Gotchas

Common Trap / Gotcha

1. Segment tree array size: Use $4n$, not $2n$ or n for the recursive version. Iterative uses exactly $2n$.

Common Trap / Gotcha

2. Lazy pushDown before recursing: Always call `pushDown` before visiting children in any lazy operation. Missing it causes queries to see stale node values.

Common Trap / Gotcha

3. Sparse table is static: Any update invalidates it. Rebuild costs $\mathcal{O}(n \log n)$ — use a segment tree instead if updates are needed.

Common Trap / Gotcha

4. BIT is 1-indexed: Calling `update(0, v)` silently does nothing. Off-by-one errors are the #1 BIT bug.

Common Trap / Gotcha

5. Mo's requires offline queries: If queries arrive online (each answer depends on the previous), Mo's cannot be used directly.

Common Trap / Gotcha

6. Persistent seg tree node limit: Each update creates $\mathcal{O}(\log n)$ nodes. For $q = 10^5$ updates on $n = 10^5$ with $\log n = 17$: allocate at least $n + q \cdot 2 \log n \approx 3.4 \times 10^6$ nodes. Pre-allocate or use a node pool.